目录

2 进程与线程管理 2
2.1 概述 2
2.2 核心设计思想 3
2.3 关键数据结构 4
2.4 进程与线程创建 5
2.5 程序执行 6
2.6 退出与资源回收 7
2.7 进程组与会话 8
2.8 权限与进程控制 10
2.9 本章小结 12
3 调度与同步机制 13
3.1 概述 13
3.2 任务调度机制 14
3.3 阻塞与唤醒机制 17
3.4 阻塞与唤醒的具体应用 19
3.5 Futex 同步机制 21
3.6 本章小结 23

2

2.1

2.1.1 StarryOS

StarryOS `Thread` `ProcessData`

StarryOS `fork` `clone` `clone3` `execve` `exit` `waitpid` pidfd

moss Linux

2.1.2 base os

“ ” StarryOS

`clone3` pidfd `execve` `execveat` `MS_NOEXEC`

`waitpid` “ ” “ ” `waitid` `siginfo_t` UID/GID capability `kill` ptrace pidfd `waitid` `/proc`

2.1.3 moss

moss StarryOS

	StarryOS	moss
	`Thread` `ProcessData`	/ namespace
	`clone3` `execve`	clone ABI `execveat`
	`waitpid`	`waitid` pidfd
	pidfd	ptrace

2.2

StarryOS moss

2.2.1

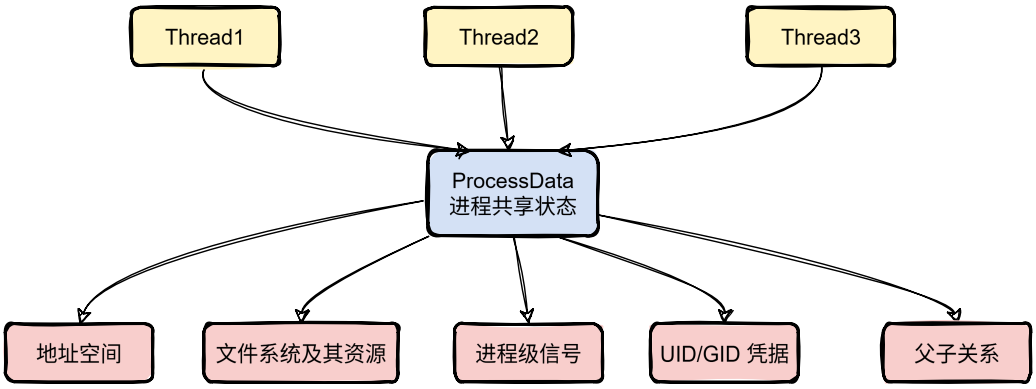
moss StarryOS

`Thread` `ProcessData`

`ProcessData` clone

`Thread`

`ProcessData` clone exec exit



2-1 Thread ProcessData

2.2.2

moss

- `ProcessData`
- parent children wait
- PID PGID SID

“ ” “ ”

“ ” moss

2.2.3

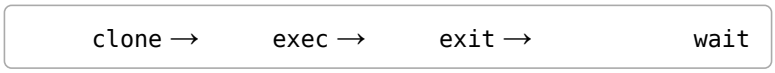
Linux UID real effective saved real ID

effective ID saved ID GID

moss UID/GID `ProcessData` `setreuid` `setresuid`

ptrace

2.2.4



exec PID
exit wait moss

2.3

2.3.1 Thread ProcessData

`Thread` `ProcessData`

```
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    uid: AtomicU32,
    euid: AtomicU32,
    suid: AtomicU32,
}
```

2-1

`Arc<ProcessData>`

`Thread` moss

2.3.2

StarryOS moss

`ProcessWaitStatus`

```
pub enum ProcessWaitStatus {
    // 被信号暂停，但进程仍然存在
    Stopped(Signo),
    // 从停止状态恢复
    Continued(Signo),
}
```

2-2

Process ProcessData

“ ” “ ” waitpid waitid

2.3.3

TID PID PGID SID moss

TASK_TABLE	TID	
PROCESS_TABLE	PID	
PROCESS_GROUP_TABLE	PGID	
SESSION_TABLE	SID	

2-2

moss

PGID SID

2.4

2.4.1 fork clone

fork moss x86_64 sys_fork sys_clone

SIGCHLD fork ProcessData

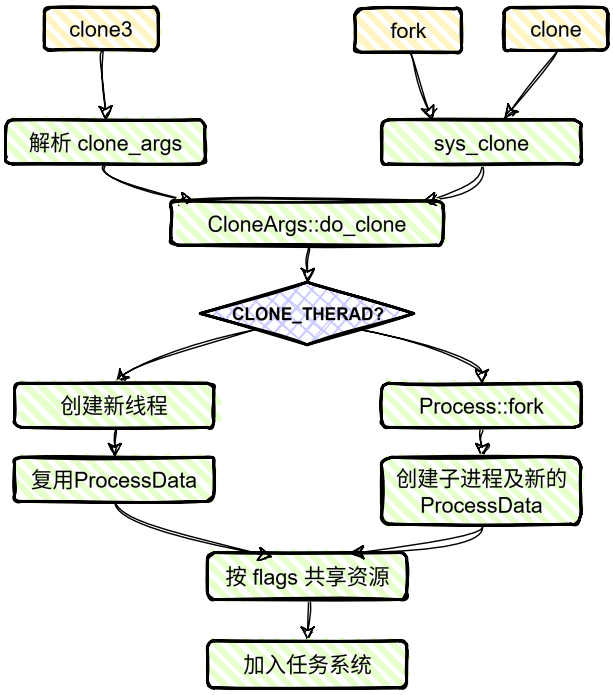
clone clone3

CloneArgs CloneArgs::do_clone

CLONE_THREAD ProcessData Process::fork

ProcessData

CLONE_VM CLONE_FILES CLONE_FS CLONE_SIGHAND



2-2 fork clone clone3

CLONE_THREAD	ProcessData
CLONE_VM	
CLONE_FILES	
CLONE_FS	
CLONE_SIGHAND	

2-3 clone

CLONE_THREAD

2.4.2

moss clone

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

2-3 clone

clone3 moss

2.4.3

pidfd

pidfd

moss

“ PID ”

CLONE_VFORK namespace

2.5

2.5.1 execve

StarryOS execve ELF

CLOEXEC exec

PID

MS_NOEXEC

2.5.2 execveat

execveat dirfd

AT_EMPTY_PATH

AT_SYMLINK_NOFOLLOW

execve

execveat AT_EMPTY_PATH

Linux execve

moss execve execveat execve_common

moss

```rust
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

2-4

2.5.3 exec

Linux exec

moss exec

2.6

2.6.1

robust futex clear_child_tid

ProcessData

moss

ptrace `wait`

2.6.2 exit_group

2.6.1 `exit_group`

moss

2.6.1

`exit` `exit_group`

2.6.3

StarryOS `children` init

→ init →

PID

init `wait`

2.7

2.7.1 PID PGID SID

PID PGID

SID PID PGID PID SID

Shell

moss StarryOS `Process` PID `ProcessGroup`

`ProcessGroup` PGID `Session` `Session` SID

```rust
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

2-5

```rust
// 先按 PID 找到进程对象
let proc = get_process_data(pid)?.proc;
let pid  = proc.pid();
// 沿所属关系取得进程组和会话标识
let pgid = proc.group().pgid();
let sid  = proc.group().session().sid();
```

2-6 PID PGID SID

`ProcessGroup::processes` moss `PROCESS_TABLE` `PROCESS_GROUP_TABLE` `SESSION_TABLE` PID PGID SID

2.7.2

StarryOS `setpgid` `getpgid` PGID

PGID

moss PGID PGID PID

`getpgid`

PGID

PGID

2.7.3

`setsid` moss `Session` `ProcessGroup`

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

2-7

PID PGID SID

2.8

```
pub struct ProcessData {
    uid: AtomicU32,  // real UID: 进程的实际身份
    euid: AtomicU32, // effective UID: 当前权限判断使用
    suid: AtomicU32, // saved UID: 保留可恢复的身份
}

let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}

if ruid != UID_GID_NO_CHANGE {
    proc_data.set_real_uid(ruid);
}
if euid != UID_GID_NO_CHANGE {
    proc_data.set_euid(euid);
}
if suid != UID_GID_NO_CHANGE {
    proc_data.set_suid(suid);
}
```

2-8 UID

effective UID 0

effective UID/GID `kill` ptrace attach

capability effective permitted inheritable bounding

2.8.2 ptrace

moss ptrace `TRACEME` `ATTACH` `DETACH` wait

attach

wait

`PTRACE_SYSCALL` `PTRACE_SINGLESTEP`

siginfo `EIO` GDB

2.8.3 namespace

moss PID UTS User namespace inode namespace init PID

clone unshare PID UID

namespace

2.9

moss StarryOS `Thread` `ProcessData` real effective saved ID clone exec exit wait

moss `clone3` ABI `execveat`

ptrace

vfork exec namespace ptrace

StarryOS

3

CPU moss
StarryOS Futex

3.1

3.1.1 StarryOS

StarryOS `axtask` `axsched`
RR CPU
CPU
StarryOS Future Waker I/O
Futex WAIT
WAKE robust list

3.1.2 base os

Futex Waker
Waker

`FUTEX_WAIT_BITSET`

`waitid` pidfd

3.1.3 moss

moss StarryOS RR

	StarryOS	moss
	RR CPU	
	Future Waker	PollSet
		TimerKey
Futex	WAIT WAKE	
	`waitpid`	`waitid` pidfd

3-1

moss
Futex

3.2

3.2.1

- `Running` CPU
- `Ready`
- `Blocked`
- `Exited`

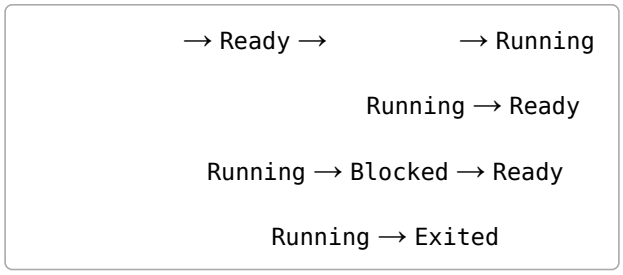
```rust
pub enum TaskState {
    Running = 1, // 正在 CPU 上执行
    Ready   = 2, // 可以运行，正在队列中等待
    Blocked = 3, // 正在等待事件，暂时不能运行
    Exited  = 4, // 已经结束，等待资源回收
}

pub struct TaskInner {
    // 调度器通过该字段判断任务当前能否运行
    state: AtomicU8,
    // ...
}
```

3-1

`state` Futex

`Blocked` `Ready`



3-2

`Ready` `Running` CPU

`Ready` `Blocked`

moss Futex

`Blocked` `Ready` moss

3.2.2

RR CPU CPU

`Ready` 5

CPU

scheduler_timer_tick



CPU

CPU idle

3.2.3

resched CPU

idle CPU

```
fn resched(&mut self) {
    // 从就绪队列选择任务；队列为空时运行 idle 任务
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将选中的任务设为正在运行
    next.set_state(TaskState::Running);

    // 保存当前任务现场，恢复下一个任务现场
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

3-3

switch_to Running “ ”

TaskContext

CPU TaskContext

moss TaskContext

UserContext

3.2.2

→ → → →

3.2.4 CPU

CPU cpumask CPU
CPU 0 CPU 2 CPU
CPU cpumask
CPU CPU

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        // 在所有 CPU 编号之间轮转
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        // 只选择亲和性掩码允许的 CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

3-4 CPU
CPU CPU
CPU
CPU moss
CPU 1
Linux sched_getaffinity sched_setaffinity

```
// 用户位图中的每一位对应一个 CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新的亲和性范围
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范围内时，将任务迁移到允许的 CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

3-5 CPU

CPU CPU

moss

`sched_yield` CPU moss

3.3

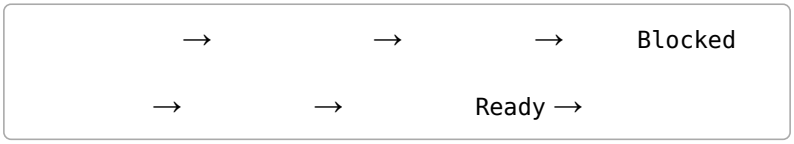
3.3.1

Future Waker moss

`Blocked`

`Blocked` `Ready`

CPU



3-6

3.3.2 Waker PollSet

Waker StarryOS

Waker `wake()` moss

timerfd pidfd Futex

`PollSet` Waker eventfd timerfd pidfd

moss `has_waiters`

Waker

```rust
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者时，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本轮登记的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正唤醒时已经不再持有内部锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

3-7 PollSet

`take_wakers` Waker `wake()`

PollSet

PollSet “ ”

Futex

3.3.3

`block_on` Future Future `Pending` `block_on`

`Blocked` Waker `Ready` CPU

Future

```rust
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先记录通知已经到达
        *self.woke.lock() = true;
        // 再把阻塞任务放回合适的运行队列
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    // 没收到通知，当前任务可以安全进入 Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    // 通知先到，不再阻塞，只让其他任务先运行一次
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

3-8 Waker block_on

`blocked_resched` `Blocked` `unblock_task` `Ready` CPU Future moss Waker `woke` CPU

moss Waker `interruptible` Future `EINTR`

3.3.4

Waker

Waker moss PollSet Futex 3.5

3.4

3.3 “ ” “ ”

3.4.1

“ ”

moss

FUTEX_WAIT_BITSET

nanosleep clock_nanosleep

TIMER_ABSTIME moss

EINTR

3.4.2

StarryOS

moss BTreeMap deadline seq

TimerKey alarm task

```
struct TimerKey {
    // 定时事件应当触发的时刻
    deadline: Duration,
    // 区分截止时间相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    // 为新事件生成不重复的序号
    next_seq: u64,
    // 按到期顺序保存全部定时事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

3-9

moss seq Key

Key Key

alarm

task

TimeManager

ITIMER_REAL SIGALRM ITIMER_VIRTUAL

ITIMER_PROF

3.4.3

ProcessData

wait

Waker child_exit_event

```rust
// 先检查是否已有子进程状态可以报告
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        // 暂无结果，登记父进程的 Waker
        child_exit_event.register(cx.waker());
        // 登记后复查，覆盖同时发生的状态变化
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

3-10

3.4.4 wait pidfd

moss

Pid Pgid Any

ECHILD

WNOHANG	
WUNTRACED	
WCONTINUED	
WNOWAIT	

3-2

waitpid wait4 waitid siginfo_t

moss waitid PID

pidfd

pidfd moss pidfd waitid

PID WNOWAIT

3.5 Futex

3.5.1

Futex

moss FutexKey Futex

Futex

Futex owner_dead

moss

```
pub struct WaitQueue {
    // 每个等待项依次保存:
    // 通知状态、唤醒位掩码和任务唤醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

3-11 Futex

AtomicBool notified u32 bitset PollSet

“ wake ”

3.5.2 WAIT WAKE

moss FUTEX_WAIT

FUTEX_WAKE EAGAIN

Future FUTEX_WAKE

notified PollSet

wake

```
// 第一次检查: 通知可能已经先于登记到达
if notified.load(Ordering::Acquire) {
    return Poll::Ready(());
}

// 条件未满足, 登记当前任务的 Waker
poll.register(cx.waker());

// 第二次检查: 覆盖"检查与登记之间"的竞争窗口
if notified.load(Ordering::Acquire) {
    Poll::Ready(())
} else {
    Poll::Pending
}
```

3-12 Futex Waker

notified Waker

Waker notified

moss Waker

3.5.3

notified FUTEX_WAIT_BITSET

FUTEX_WAKE_BITSET bitset mask

`FUTEX_REQUEUE` Futex `FUTEX_CMP_REQUEUE` `EAGAIN` `notified` bitset PollSet Waker pthread

3.5.4

Futex moss `interruptible(timeout(...))` `ETIMEDOUT` `EINTR` `notified` `FUTEX_WAIT` `FUTEX_WAIT_BITSET` moss Linux `FUTEX_CLOCK_REALTIME`

LoongArch64 `futex_wait` pthread cancellation

3.5.5 robust list

moss Futex `set_robust_list` robust list Futex moss `futex_offset` Futex `owner_dead` `EOWNERDEAD` `ROBUST_LIST_LIMIT` `clear_child_tid` Futex `pthread_join`

3.6

StarryOS RR moss PollSet `TimerKey` moss Futex robust list `waitid` pidfd